



UC San Diego

Design principles for integrating external workflow systems with the CMS Global Pool

Marco Mascheroni

University of California San Diego

CMS Spring O&C Week 2026 – March 17th, CERN

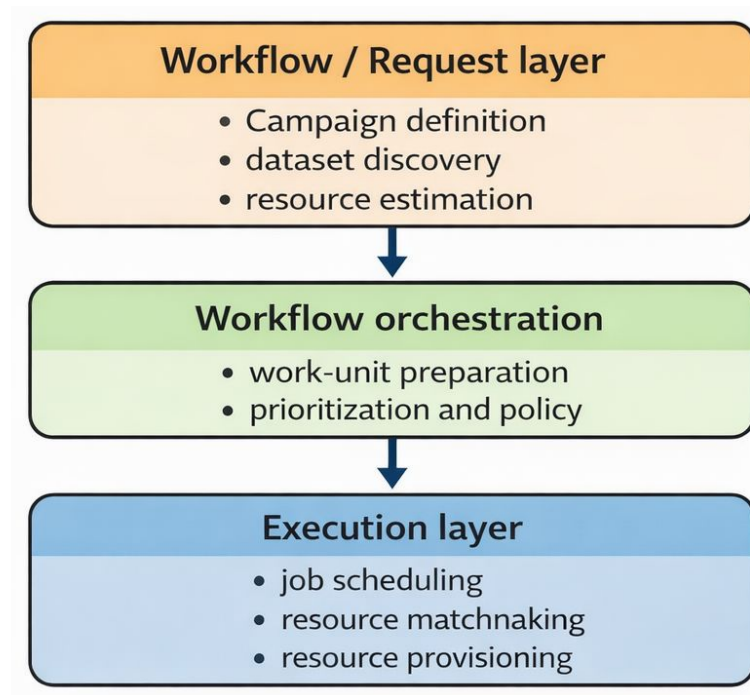
Context

- We are evaluating multiple directions for the HL-LHC workflow management system
 - DIRAC
 - PanDA
 - In-house CMS architecture
- Each system provides similar capabilities:
 - workflow and work unit orchestration
 - data management integration
 - job execution and scheduling

The real architectural question is not which system to choose, **but where the architectural boundary should be**

The Workflow Stack: Theory

- Natural WM layer organization
 - Workflow/request layer
 - Workflow orchestration
 - Execution layer
- Each system provides similar capabilities
 - workflow and work unit orchestration
 - data management integration
 - job execution and scheduling
- Different systems draw boundaries at different places



The real architectural question is not which system to choose, **but**
where the architectural boundary should be

The Workflow Stack: Practice

Where does **job scheduling** happen?

CMS

WORKFLOW / REQUEST

- McM
- ReqMgr



WORK UNIT ORCHESTRATION

- GlobalWorkQueue
- WMAgent



SCHEDULING BOUNDARY

EXECUTION

- GlideinWMS
- HTCondor

ATLAS

WORKFLOW / REQUEST

- ProdSys



WORK UNIT ORCHESTRATION

- JEDI
- PanDA Server



SCHEDULING BOUNDARY

EXECUTION

- Harvester
- PanDA Pilot

LHCb

WORKFLOW / REQUEST

- LbMCSubmit
- Production Management



WORK UNIT ORCHESTRATION

- Transformation System
- WorkflowTaskAgent



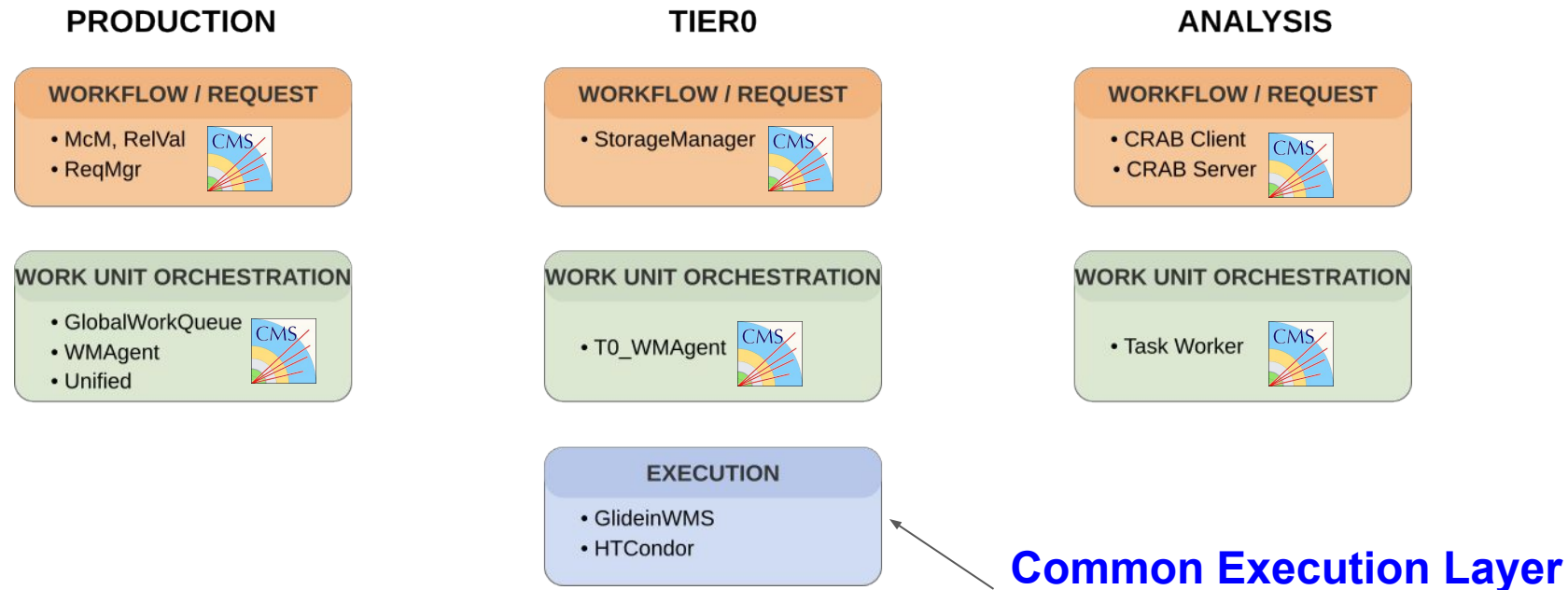
SCHEDULING BOUNDARY

EXECUTION

- Workload Management System (job scheduling)

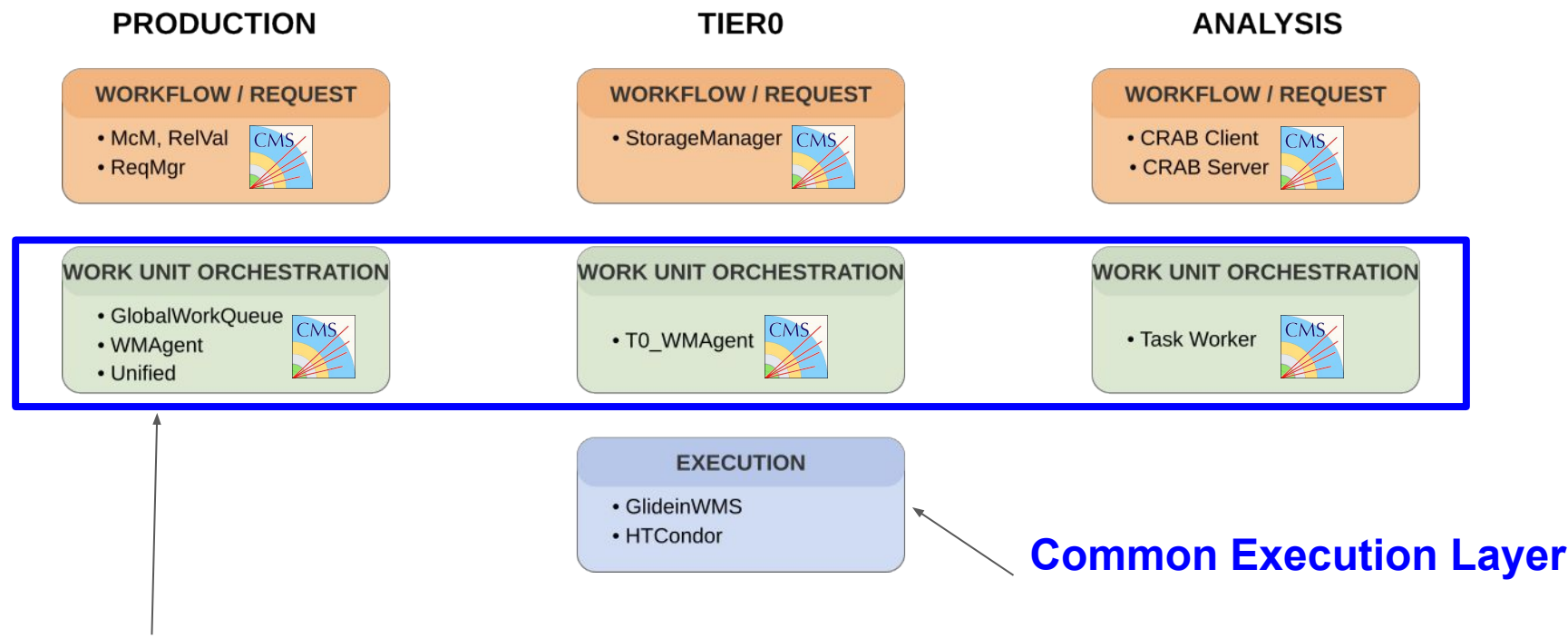


Current CMS Workflow Orchestration



- CRAB implements a stateless work-unit orchestration layer
- Tier-0 and Production rely on WMAgent components
 - Operational logic is tightly coupled with **job-level state**

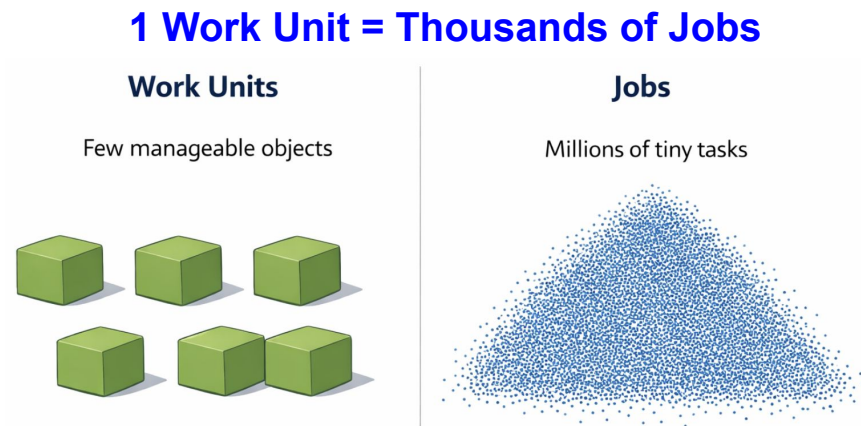
Current CMS Workflow Orchestration



- Opportunity for a **common work-unit orchestration layer**

Where complexity actually lives

- Workflow and workload management is operated at different scales
- Systems that manage job payload must handle:
 - millions of entries
 - large scheduling state
 - high operational complexity
- Systems that operate on work units handle:
 - orders of magnitude less state
 - simpler orchestration problems



Operational complexity grows at the job level:

Execution and work-unit orchestration layers should remain separated

CMS Execution Layer

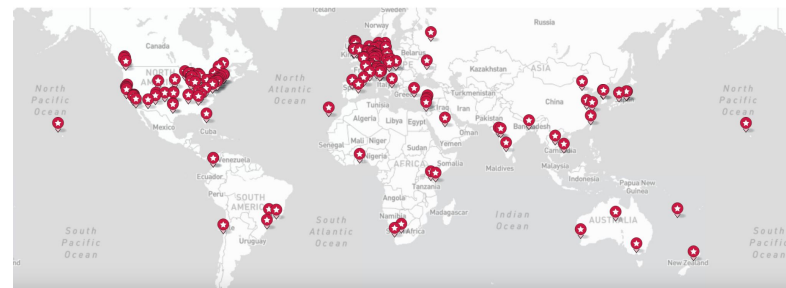
- **GlideinWMS** is used by >20 experiments without constraining development.
 - OSG only encloses a **dozen of different experiments** and up to about **100 individual research groups** across all the science use-cases

HTCondor + GlideinWMS
form a mature ecosystem used
across many scientific communities

HTCSS - Trusted to Manage the Workloads of 100s of Academic, Government and Industry Institutions

Map of 175 Pools Utilizing HTCSS (user reported)
Add your organization to help us report our impact!

Add Organization +

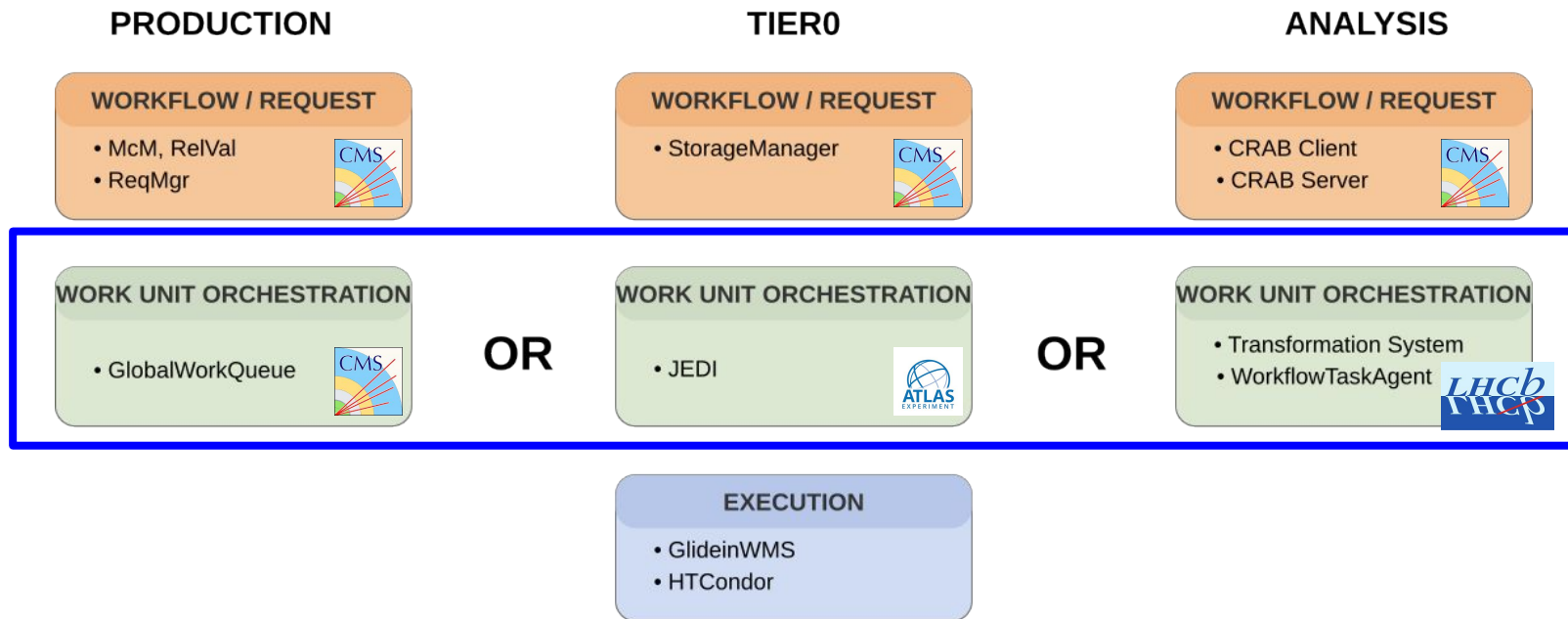


Clean Architecture Principles

- Prioritize clean architecture and long term maintainability
 - For systems operating over 10–15 years (as we learned)
Operational cost dominates development effort
- **Avoid split-brain** scheduling systems
 - Provisioning decisions, matchmaking, and site definitions must live in **one place**
 - Two schedulers engines:
 - duplicated matchmaking
 - conflicting provisioning logic
 - operational complexity
 - difficult debugging

A clean separation between orchestration and execution reduces operational complexity and avoids split-brain scheduling systems

The three options on the table

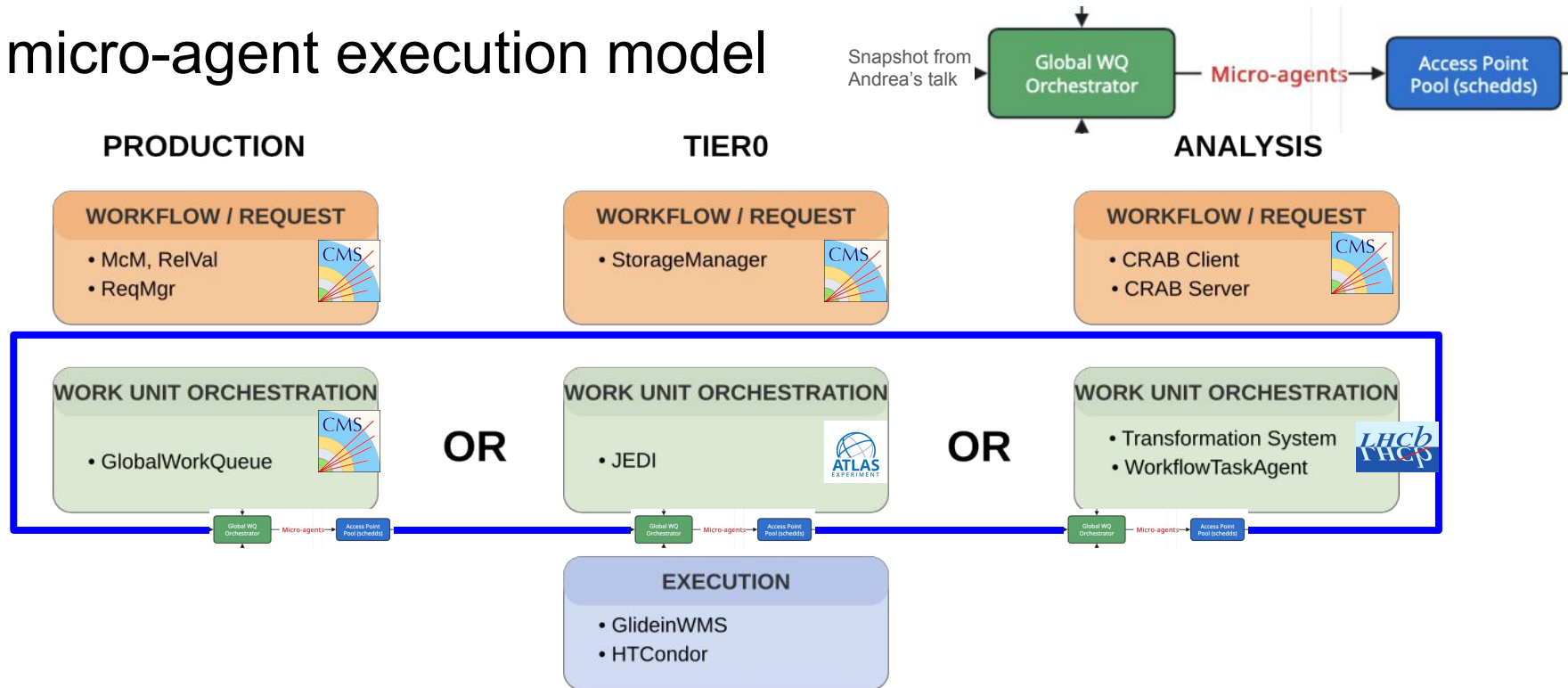


CMS can integrate external workflow technologies while preserving the **Global Pool execution layer**

- Reuse where it makes sense: **workflow preparation** and campaign management
- Preserve where CMS already has strong infrastructure: scheduling, resource provisioning

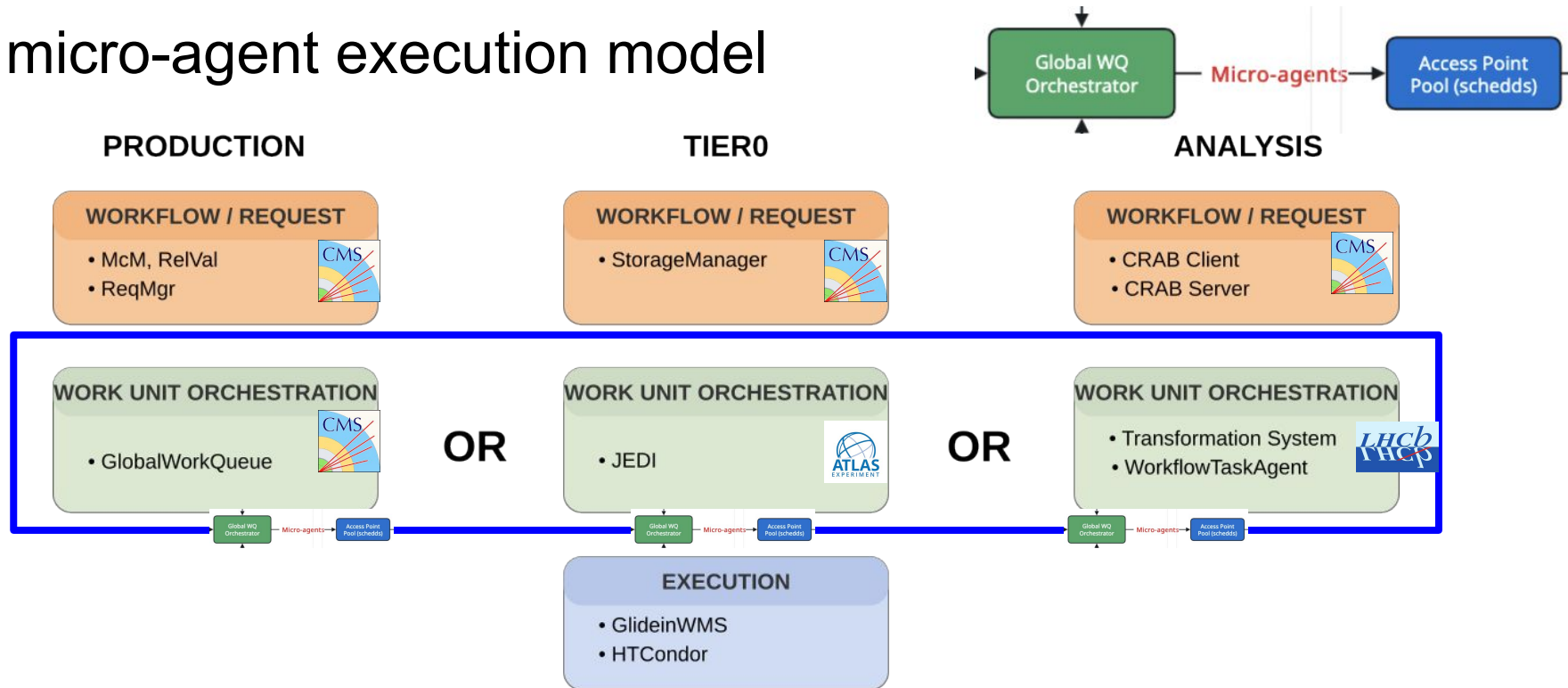
External workflow systems **integrate at the work-unit orchestration layer**.

The micro-agent execution model



If the scheduling boundary remains at the Condor layer, we only need a thin layer translating work units into jobs.

The micro-agent execution model



The micro-agent execution model acts as a reusable layer that translates work units into jobs and manages their execution.

A Low-Risk Evolution Path

- Step 1
 - Replace WMAgent with micro-agent execution model
- Step 2
 - Replace the Global WorkQueue orchestration layer
- Step 3
 - Replace request management components (ReqMgr)

Throughout the transition, we can keep:

- HTCondor Global Pool
- GlideinWMS provisioning
- CRAB and Tier-0 infrastructures
 - They can be transitioned in a second phase

This allows CMS to modernize the workflow control plane while preserving the proven execution substrate

Conclusions

- CMS already operates a **mature execution substrate** (HTCondor + GlideinWMS)
- External workflow systems should provide strong capabilities for **workflow preparation** and **work unit orchestration**
- The largest operational complexity lies at the **job scheduling scale**

It is fundamental to separate workflow orchestration from job execution and scheduling

- Workflow tools operate at the **work-unit level**
- Execution infrastructure operates at the **job level**

**CMS can integrate external workflow technologies
while preserving the Global Pool execution layer**